

- Библиотеки
 - Виды библиотек
 - Вывод атрибутов элемента
 - Встроенные библиотеки
 - Сторонние библиотеки
 - Популярные сторонние библиотеки
 - Виртуальное окружение
 - Модули
 - Где Python ищет модули
 - Специальные переменные модуля
 - `__name__`
 - `__all__`
 - Пакеты
 - Правила организации модуля

Библиотеки

Библиотека (модуль) – файл с кодом, содержащий функции, классы и переменные, которые можно использовать в своих программах. Это готовые наборы ранее написанного и отлаженного кода.

Импорт

```
# Прямой
import math

# Отдельные функции
from math import sqrt, floor

# Переименование
import math as m
```

Виды библиотек

1. Встроенные (включенные в стандартную библиотеку Python и доступные сразу после установки языка). `random, math, os, sys, json, collections, re, csv, unittest, datetime`
2. Сторонние библиотеки (созданы другими разработчиками, требуют дополнительной установки). `requests, pandas, numpy, matplotlib, beautifulsoup, fastapi, django, flask, selenium, tensorflow, pytorch`
3. Собственные (созданные вами для организации своего кода)

Вывод атрибутов элемента

```
import random
attributes = dir(random)

print(attributes[:10])
```

Комплектующие дистрибутива Python:

- Интерпретатор
- IDLE (встроенная среда разработки)
- pip
- Стандартная библиотека
- Документация и утилиты

Дистрибутив - сборка, позволяющая установить ЯП на ПК `python-3.12.4-amd64.exe`.

Встроенные библиотеки

Встроенная библиотека (стандартная библиотека) – это набор модулей, включённых в дистрибутив Python и готовых к использованию без дополнительной установки.

1. math

```
import math

math.pi
math.e
```

```
math.sin(math.pi / 4):.4f
math.cos(math.pi / 4):.4f
math.factorial(5)
math.gcd(12, 18) # наибольший общий делитель 12 и 18: 6
```

2. random - генерация случайных чисел и выбор случайных элементов

```
import random

random.randint(1, 10)
random.random():.4f # случайное число с плавающей точкой от 0 до 1: 0.3528

# Выбор случайного элемента последовательности
fruits = ["яблоко", ...]
random.choice(fruits) # яблоко

# Перемешивание последовательности
numbers = [1, 2, 3, 4, 5]
random.shuffle(numbers) # [3, 1, 5, 2, 4]
```

3. datetime - предоставляет классы для работы с датой и временем

```
import datetime

# Текущая дата и время
now = datetime.datetime.now() # 2026-06-28 14:39:25.225090

# Задание конкретной даты
specific_date = datetime.date(2026, 12, 31) # 2026-12-31

# Разница между датами
today = datetime.date.today()
new_year = datetime.date(today.year + 1, 1, 1)
days_until_new_year = (new_year - today).days

print('До нового года', days_until_new_year)

# Форматирование даты
formatted_string = now.strftime("%d.%m.%Y %H:%M") # 15.07.2023 15:42
```

4. os - набор функций для взаимодействия с операционной системой

```
import os

# Получение текущей директории
current_dir = os.getcwd()

# Список файлов и папок в директории
```

```
files = os.listdir('.')

# Информация о системе
sys_info = os.name

# Проверка существования файла/директории
file_exists = os.path.exists('cases.md')
```

5. json - предоставляет функции для работы с данными в JSON-формате

```
import json

# Словарь Python
person = {
    "name": "Иван",
    "age": 30,
    "city": "Москва",
    "languages": ["Python", "JavaScript", "SQL"]
}

# Преобразование Python-объекта в JSON
python_to_json = json.dumps(person, ensure_ascii=False, indent=4)

# Преобразование JSON в Python-объект
json_object = '{ "name": "Igor", "age": 29, "is_admin": True }'
json_to_python = json.loads(json_object)
```

`json.dumps` превращает Python-объект в JSON-строку. Это необходимо, чтобы отдать данные по API, сохранить в файл или передать в Django-шаблон.

- `python_to_json` рекурсивно преобразуется в json
- `ensure_ascii=False` отображает кириллицу
- `indent=4` даёт отступы

`json.loads(string)` (load string) берёт json и превращает в Python-объект. Парсит (разбирает) строку по правилам JSON и строит из неё структуры данных Python.

`json.load(file_object)` - читает JSON из файла. На вход подаётся открытый файл (результат `open(...)`).

6. Collections - удобные структуры данных поверх базовых. В модуле также есть `defaultdict`, `namedtuple`, `deque` и другие.

```
from collections import Counter
```

```
text = "Programming on Python"
character_count = Counter(text.lower()) # # Counter({'o': 3, 'n': 3, 'p': 2, 'r': 2, 'g': 2, 'm': 2, ' ': 2, 'a': 1, 'i': 1, 'y': 1, 't': 1, 'h': 1})

print("Three most frequently chars")
for char, count in character_count.most_common(3):
    print(f"'{char}': {count}")

# 'o': 3
# 'n': 3
# 'p': 2
```

Сторонние библиотеки

Сторонние библиотеки – это модули Python, которые не входят в стандартную библиотеку и разрабатываются независимыми разработчиками или организациями.

pip (package installer for python) - система управления пакетами, которая используется для установки и управления программными пакетами на языке Python. Pip скачивает пакеты из Python Package Index (PyPI), а также умеет обновлять, удалять, фиксировать версии зависимостей проекта.

PyPI (Python Package Index) – это хранилище программного обеспечения для ЯП Python. С его помощью можно найти нужный программный блок и внедрить его в свой проект. PyPI помогает находить и устанавливать ПО, разработанное сообществом Python. Авторы пакетов используют PyPI для распространения своего программного обеспечения.

Популярные сторонние библиотеки

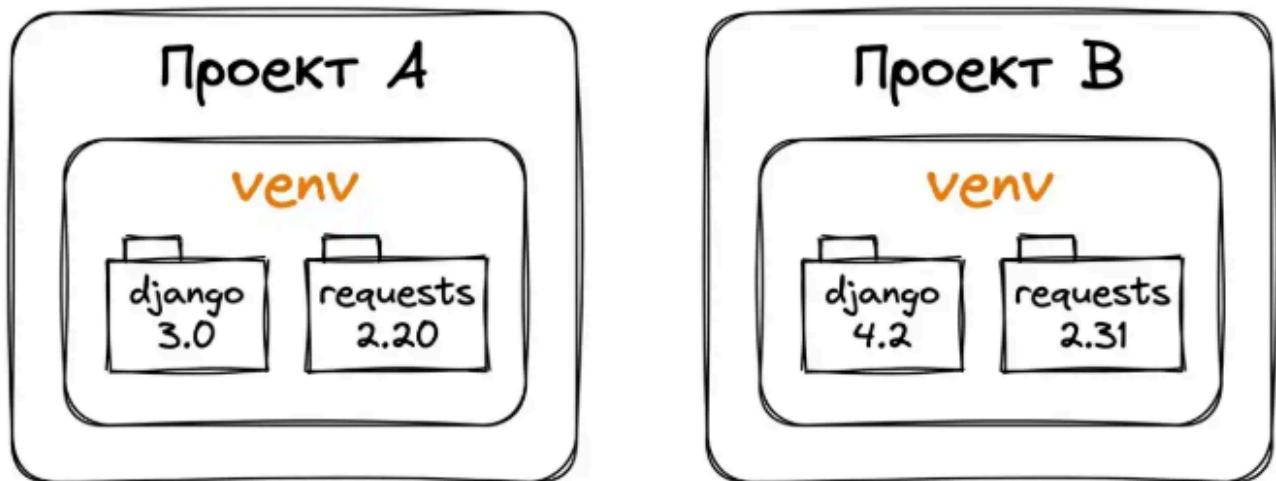
Библиотека	Описание	Назначение
requests	Удобная работа с HTTP-запросами	Взаимодействие с веб-ресурсами, API
pandas	Удобная работа с табличными данными	Обработка и анализ табличных данных
SQLAlchemy	ORM для работы с БД	Взаимодействие с SQL и БД

Библиотека	Описание	Назначение
pillow	Обработка изображений	Редактирование и анализ изображений
bs4	Парсинг HTML и XML	Извлечение данных с веб-страниц
django	Полнофункциональный веб-фреймворк	Крупные веб-проекты
flask	Микрофреймворк для веб-разработки	Создание веб-приложений и API
fastapi	Современный асинхронный фреймворк для создания API на Python	Быстро строить REST/JSON-API
tensorflow	Глубокое обучение и нейронные сети	Сложные модули глубокого обучения
pytorch	Глубокое обучение с динамическими вычислительными графами	Исследования в области глубокого обучения
matplotlib	Визуализация данных	Построение графиков и диаграмм
numpy	Работа с массивами и математическими вычислениями	Научные вычисления, работа с массивами
PyQt	Создание настольных приложений	Графические интерфейсы (GUI)
Pygame	Создание игр и мультимедийных приложений	Разработка 2D-игр

Виртуальное окружение

Виртуальное окружение (venv) – это изолированная среда Python, в которой можно устанавливать свои пакеты, не влияя на другие проекты или системный Python.

Каждый проект — своё venv



Модули

Модуль — это просто файл с разрешением `.py`, содержащий код Python (функции, классы, переменные), который можно импортировать и использовать в других программах.

Импорт всего модуля - когда из одного модуля нужно достать много функций, при этом видеть, откуда они пришли.

Импорт конкретных элементов - обращение без префикса хорошо для пары часто вызываемых функций и плохо, когда через сотню строк кода забываешь источник.

Импорт с переименованием - когда имя модуля слишком длинное или конфликтует с названием переменной.

Импорт всех элементов - не рекомендуемый способ, добавляет все функции модуля в пространство имён: теряется источник имён, легко получить конфликт. Нормально только в REPL и иногда в тестах.

Где Python ищет модули

Когда мы пишем `import mymath`, Python ищет файл `mymath.py` по списку директорий из `sys.path`. По умолчанию туда входят:

- Директория запускаемого файла (или текущая директория из REPL)
- Стандартная библиотека Python (`math`, `os`)
- `site-packages` - папка, куда `pip install` устанавливает сторонние пакеты

Поиск прекращается на первой найденной директории. Если рядом со скриптом лежит файл с тем же именем, что и стандартный модуль, Python подхватит наш файл вместо системного. Классическая ловушка: создать кастомный модуль `random` и перетереть встроенный `random`.

Специальные переменные модуля

Модули в Python имеют несколько специальных переменных.

`__name__`

`__name__`: модуль как программа vs зависимость. Внутри Р у каждого М есть переменная `__name__`, когда М импортируют, в ней лежит его имя `mymath`. Когда М запускают напрямую `python mymath`, Р кладёт в неё специальное значение `__main__`, – это позволяет внутри модуля написать блок "делай это только при прямом запуске".

```
"""Модуль с математическими функциями"""

PI = 3.14159

def add(a, b):
    return a + b

def divide(a, b):
    if b == 0:
        raise ValueError("Деление на ноль невозможно")
    return a / b

# Этот блок выполняется только при прямом запуске
if __name__ == "__main__":
    print(f'PI = {PI}')
    print(f'add(2, 3) = {add(2, 3)}')
```

`__all__`

`__all__`: что попадает в `from module import *`

По умолчанию `from mymath import *` импортирует все имена, которые не начинаются с подчёркивания. Если хочется явно зафиксировать публичный API модуля, добавляют `__all__` со списком имён

```
# Только эти имена попадут в from mymath import *
__all__ = ["PI", "add"]

PI = 3.14159
_INTERNAL = "не должен попасть наружу"

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def _round_helper(value):
    """Внутренний помощник, не для публичного использования"""
    return round(value, 2)
```

Через `from mymath import *` пришли только `PI` и `add`, для импорта остальных нужно импортировать конкретно `from mymath import subtract`.

Пакеты

Пакет - директория с файлом `__init__.py`. Когда модуль `mymath` разрастается, его можно разделить на несколько файлов и собрать в пакет. Когда выполняется команда `import mathlib`, Python видит этот `__init__.py` и понимает, что директория - это импортируемый пакет.

.Пример `__init__.py`

```
"""mathlib - пакет с математическими функциями"""

__version__ = "0.1"

# Re-export, чтобы пользователи могли писать from mathlib import add
from mathlib.basic import PI, add, subtract
from mathlib.advanced import multiply, divide
```

`mathlib/__init__.py` контролирует, что считается публичным API: имена, перечисленные через `from .basic import ...` и `from .advanced import ...`, доступны прямо как `mathlib.add`.

Если пакет разрастается, его можно делить над подпакеты со своими `init.py`.

Правила организации модуля

1. **Единая ответственность.** Каждый модуль отвечает за одну конкретную задачу: `auth`, `formatters`. Если в файле две несвязанные темы, пора разделять.
2. **Понятные имена.** Описательные, но короткие, в стиле `snake_case`: `user_interface.py`.
3. **Порядок содержимого внутри файла:**
 1. Docstring - описание модуля в тройных кавычках в начале файла
 2. Импорты - сначала стандартная библиотека, потом сторонние пакеты, потом собственные модули
 3. Константы - глобальные неизменные значения
 4. Классы и функции - основное содержимое
 5. Блок `if __name__ == "__main__":` код, который выполняется только при прямом запуске файла.
4. **Явные импорты:** `from module import specific_thing` вместо `from module import *`
5. **Приватные имена с `b`.** Функции и переменные для внутреннего пользования нужно начинать с подчёркивания (`_helper`, `_INTERNAL_CONST`). Это сигнал для других: "Не полагайтесь на это снаружи".